Test assignment plan — 13 role owners + 4 senior tracks

Companion to `docs/dashboard-test-assignments.md` (the per-role checklists) and `docs/manual-test-guide.md` (how to run the stack and what to look for).

Where the project actually stands

Backend tests	930 passing
Dependency CVEs	zero, backend and frontend (`make audit-deps`)
WASA cybersecurity track	blockers closed; one Medium open (CSP `unsafe-inline`)
WASA ABDM track	**not assessable** — `hip/`, `hiu/`, `consent/`, `nhcx/` are empty
Screens	34 workspaces, 47 role/workspace pairs across 13 roles

What automated testing already covers: every screen loads, every API call on load succeeds, every role lands where it should. `e2e/dashboards.smoke.mjs` runs that on real Keycloak logins in CI.

What it deliberately does not cover, and is therefore what this exercise is for: mutating workflows. Raising an indent and approving it, dispensing a prescription, resolving a grievance, confirming a saved value survives a refresh. Every defect this project shipped rendered perfectly — `/radiology` was a title-only shell for weeks, procurement's approval queue was empty rather than broken, ABHA verification 401'd silently for months. Loading is not working, and only a human clicking the primary button finds that out.

Why assignment is by ROLE, not by screen

The obvious cut — 34 screens across 13 people — is wrong. A screen behaves differently depending on who opens it: `/consent` is reachable by four roles, `/inventory` by three, and each must be checked for *its own* permitted actions. Splitting by screen would test each one once and prove nothing about authorization.

So the unit stays "one role, one owner". The problem that creates is load:

Role	Screens
admin	11
doctor	10
receptionist	5
nurse	4
pharmacist, auditor	3
labtech, radiology, supervisor, hod	2
emergency, patient, superadmin	1

An 11× spread. The fix is not to re-cut the roles — it is to give the light roles the cross-cutting work that no single role can test anyway.

Assignments

Tier 1 — heavy roles (senior)

Owner	Role	Screens	Also owns
Kunal Sharma (S1)	`dev.admin`	11	Track A: cross-role authorization matrix
Suprita Panda (S2)	`dev.doctor`	10	Track B: the clinical order journey

Tier 2 — mid roles

Owner	Role	Screens	Also owns
Vani Khaiwal (D3)	`dev.receptionist`	5	—
Aditya Kumar (D4)	`dev.nurse`	4	—
Ajay Singh (D5)	`dev.pharmacist`	3	Inventory's five tabs (only two load on mount)
Aman Saxena (D6)	`dev.auditor`	3	—

Tier 3 — light roles, paired with a cross-cutting track

Owner	Role	Screens	Also owns
Priyanshu Swarup (D7)	`dev.labtech`	2	Maintenance null-downtime check
Riya Raj (D8)	`dev.radiology`	2	Radiology workflow: schedule → scan → report
Vanshika (D9)	`dev.supervisor`	2	Verify #221 against what actually exists
Ishika (D10)	`dev.hod`	2	Indent approval — HOD is the only role that can
Khushi (D11)	`dev.emergency`	1	**Track C: session and token behaviour**
Ankit Singh (D12)	`dev.patient`	1	**Track D: error-message quality sweep**
Aditya (D13)	`dev.superadmin`	1	Confirm the platform boundary holds

Senior tracks in full

Track A — cross-role authorization (S1). For each of the 13 roles, attempt one forbidden route and confirm a redirect. Then — and this is the part that matters — call the same endpoint directly with that role's bearer token. A hidden menu is not access control. Anything that returns 200 to the API but hides the screen is a finding.

Track B — the clinical order journey (S2). Place a lab order as the doctor, collect the sample as `dev.labtech`, enter and verify a result, confirm it appears on `/doctor/results`, and confirm the charge reaches `/billing`. This spans four roles and is the path most likely to have a dead end, because no single-role test can see it.

Track C — session and token behaviour (D11). Idle past token expiry then act — expect clean re-auth, not a wall of 401s. Log out, press Back: the previous screen must not render with live data. Log in as one role, log out, log in as another: confirm the second session does not inherit the first. That last one has a real trap — logging out via cookie-clearing rather than the app's Logout leaves the Keycloak SSO session alive.

Track D — error-message quality (D12). Across every form reachable in any role, submit invalid data once. The message must be readable and inline. Then force an API failure (stop the backend container mid-action). The toast must not show raw JSON, a Pydantic error array, stack text, or an internal ID. This is a WASA item: error handling that leaks internals is a reportable finding, and it is invisible to every automated test we have.

Onboarding — identical for all 13

```
git clone git@github.com:IUI-Solutions-HealthDoc/healthdoc.git

cd healthdoc

git checkout staging

cp .env.example .env        # defaults work for local; no secrets needed

make setup                  # ~5 min first run
```

`make setup` must end with `Seeded development facility and 13 authenticated users`. If it stops before that line, the accounts do not exist and every login will fail — do not proceed, report it.

Then:

- **App:** https://localhost — accept the self-signed certificate.
- **Every account's password:** `devpass`
- Usernames and landing pages: `docs/manual-test-guide.md`.

Keep two things open the whole time:

```
make logs      # second terminal
```

and **DevTools → Network, filtered to `/api/v1`**. Most failures appear in one of those before they appear on screen.

Prerequisites

Docker Desktop (8 GB+ allocated), Git, and a browser. Nothing else — no local Python or Node needed for manual testing.

If something breaks

```
make down && make setup        # rebuild from scratch

make contract                  # does the frontend call an endpoint that exists?

docker compose -f infra/docker-compose.yml --env-file .env logs --tail=100 backend
```

What to report, and how

One GitHub issue per defect. Not a spreadsheet — an issue can be assigned, linked to a PR and closed.

Title: `[<role>] <route> — <what is wrong>`

Body must contain:

1. Role and account used
2. Exact route
3. Steps to reproduce
4. Expected vs actual
5. **The Network tab entry** — method, path, status. This is the single most useful line; "the page is broken" costs an hour that `POST /pharmacy/grn → 422` costs five minutes.

6. Backend log excerpt if there is one
7. Screenshot

Label `manual-test` plus the role name.

Three things that look like bugs and are not

Report them only if the behaviour differs from this:

- `/receptionist/registration`, `/emergency` and `/admin/abdm-sync` make **no API call on load**. The first two are forms; the third is search-driven.
- `/admin/data-protection` returns **404 on `GET /dpdp/dpo`** until a DPO is appointed. The screen must explain that state — if it shows a raw error instead, that *is* a bug.
- **ABDM enrolment answers 503.** `ABDM_PUBLIC_KEY_PEM` and sandbox credentials are not set locally, so the endpoints refuse rather than transmitting an Aadhaar number unencrypted. Expected.

Sequencing

1: everyone runs `make setup` and confirms they can log in as their role. Nothing else. Getting thirteen environments working is its own task and doing it in parallel with testing hides which is failing.

2: per-role checklists from `dashboard-test-assignments.md`.

3: senior tracks A–D, once the per-role passes have found the obvious breakage.

4: triage, fix, retest.

One note on expectations: the automated suite already proves these screens load and their calls succeed. If a day of manual testing finds nothing, that is consistent with the automation working — the value here is specifically the mutating workflows and the error-handling quality that nothing automated covers. A thin bug list is not a failed exercise; an empty one from someone who only loaded pages is.